

1 springboot启动流程

<https://www.processon.com/view/6118c685e0b34d116b7470ea>

1.1 springboot的启动类入口

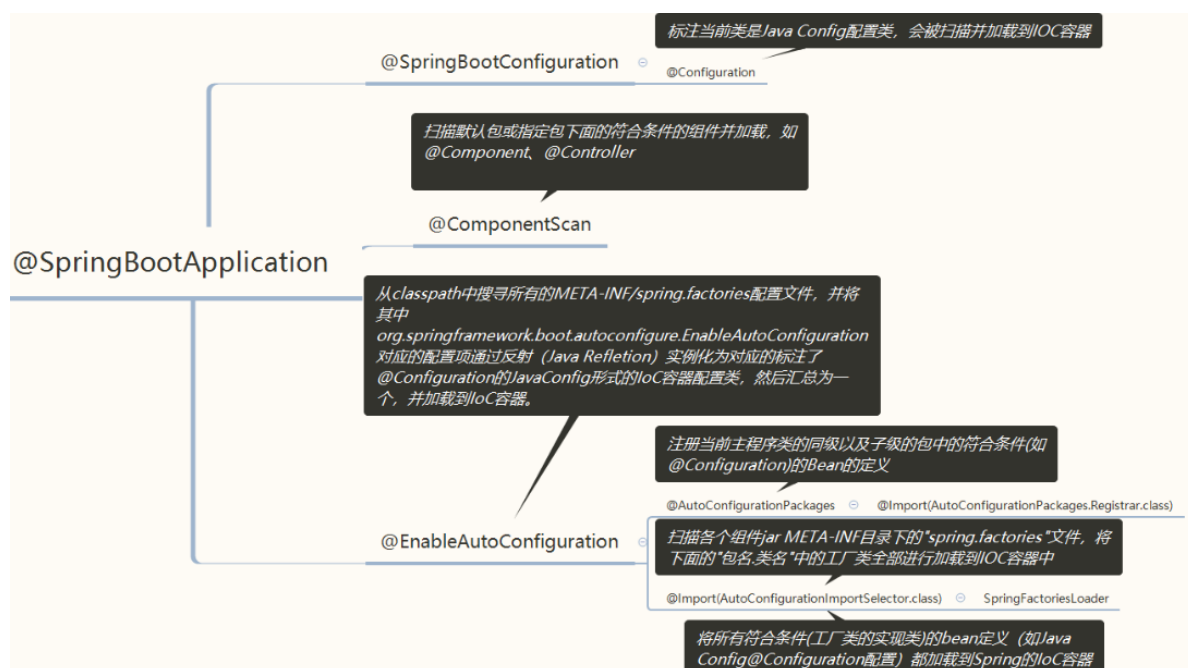
springboot有自己**独立的启动类**（独立程序）

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Annotation定义（`@SpringBootApplication`）和类定义（`SpringApplication.run`）最为耀眼，所以要揭开SpringBoot的神秘面纱，我们要从这两位开始就可以了。

1.2 SpringBootApplication注解

```
@Target(ElementType.TYPE) // 注解的适用范围，其中TYPE用于描述类、接口（包括包注解类型）或enum声明
@Retention(RetentionPolicy.RUNTIME) // 注解的生命周期，保留到class文件中（三个生命周期）
@Documented // 表明这个注解应该被javadoc记录
@Inherited // 子类可以继承该注解
@SpringBootConfiguration // 继承了Configuration，表示当前是注解类
@EnableAutoConfiguration // 开启springboot的注解功能，springboot的四大神器之一，其借助import的帮助
@ComponentScan(excludeFilters = { // 扫描路径设置
    @Filter(type = FilterType.CUSTOM, classes = TypeExcludeFilter.class),
    @Filter(type = FilterType.CUSTOM, classes = AutoConfigurationExcludeFilter.class) })
public @interface SpringBootApplication {
    ...
}
```



在其中比较重要的有三个注解，分别是：

- `@SpringBootConfiguration` // 继承了 `Configuration`，表示当前是注解类
- `@EnableAutoConfiguration` // 开启 `springboot` 的注解功能，`springboot` 的四大神器之一，其借助 `@import` 的帮助
- `@ComponentScan` (`excludeFilters = { // 扫描路径设置 (具体使用待确认)`)

1.2.1 ComponentScan注解

作用：a) 对应xml配置中的元素；

b) **(重点)** `ComponentScan` 的功能其实就是自动扫描并加载符合条件的组件（比如 `@Component` 和 `@Repository` 等）或者bean定义；

c) 将这些bean定义加载到 **IoC** 容器中。

我们可以通过 `basePackages` 等属性来**细粒度**的定制 `@ComponentScan` 自动扫描的范围，如果不指定，则**默认** `Spring` 框架实现会从声明 `@ComponentScan` 所在类的 `package` 进行扫描。

注：所以 `SpringBoot` 的启动类最好是放在 `root package` 下，因为默认不指定 `basePackages`

1.2.2 EnableAutoConfiguration注解

此注解顾名思义是可以**自动配置**，所以应该是 `springboot` 中**最为重要**的注解。

在 `spring` 框架中就提供了各种以 `@Enable` 开头的注解，例如： `@EnableScheduling`、`@EnableCaching`、`@EnableMBeanExport` 等； `@EnableAutoConfiguration` 的理念和做事方式其实一脉相承简单概括一下就是，借助 `@Import` 的支持，收集和注册特定场景相关的 **bean** 定义。

- `@EnableScheduling` 是通过 `@Import` 将 `Spring` 调度框架相关的bean定义都加载到 `IoC` 容器【定时任务、时间调度任务】
- `@EnableMBeanExport` 是通过 `@Import` 将 `JMX` 相关的bean定义加载到 `IoC` 容器【监控 `JVM` 运行时状态】
- `@EnableAutoConfiguration` 也是借助 `@Import` 的帮助，将**所有符合自动配置条件的bean**定义加载到 `IoC` 容器。
- `@EnableAutoConfiguration` 作为一个复合Annotation,其自身定义关键信息如下：

```

@SuppressWarnings("deprecation")
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@AutoConfigurationPackage【重点注解】
@Import(AutoConfigurationImportSelector.class)【重点注解】
public @interface EnableAutoConfiguration {
    ...
}

```

其中最重要的两个注解已经标注：1、`@AutoConfigurationPackage`【重点注解】2、`@Import(AutoConfigurationImportSelector.class)`【重点注解】

当然还有其中比较重要的一个类就是：`AutoConfigurationImportSelector.class`

1.2.2.1 `AutoConfigurationPackage`注解：

```

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@Import(AutoConfigurationPackages.Registrar.class)
public @interface AutoConfigurationPackage {

}

```

通过`@Import(AutoConfigurationPackages.Registrar.class)`

```

static class Registrar implements ImportBeanDefinitionRegistrar,
DeterminableImports {

    @Override
    public void registerBeanDefinitions(AnnotationMetadata metadata,
        BeanDefinitionRegistry registry) {
        register(registry, new PackageImport(metadata).getPackageName());
    }

    .....

}

```

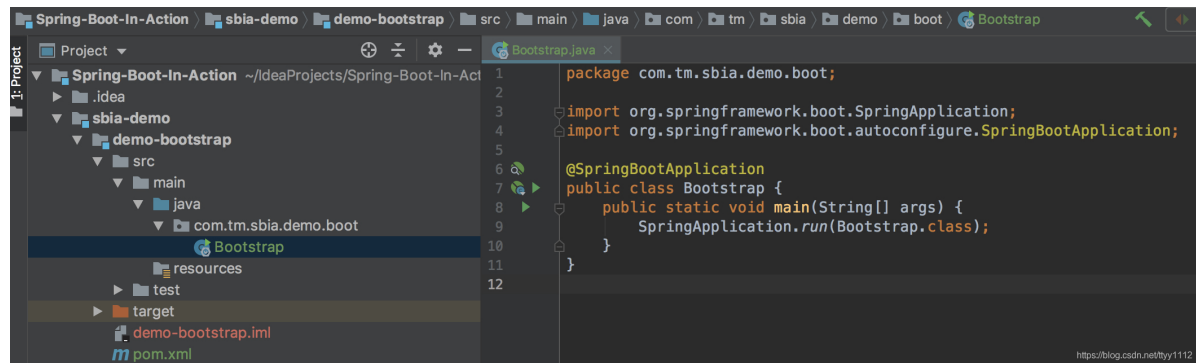
注册当前启动类的根package；

注册org.springframework.boot.autoconfigure.`AutoConfigurationPackages`的BeanDefinition。

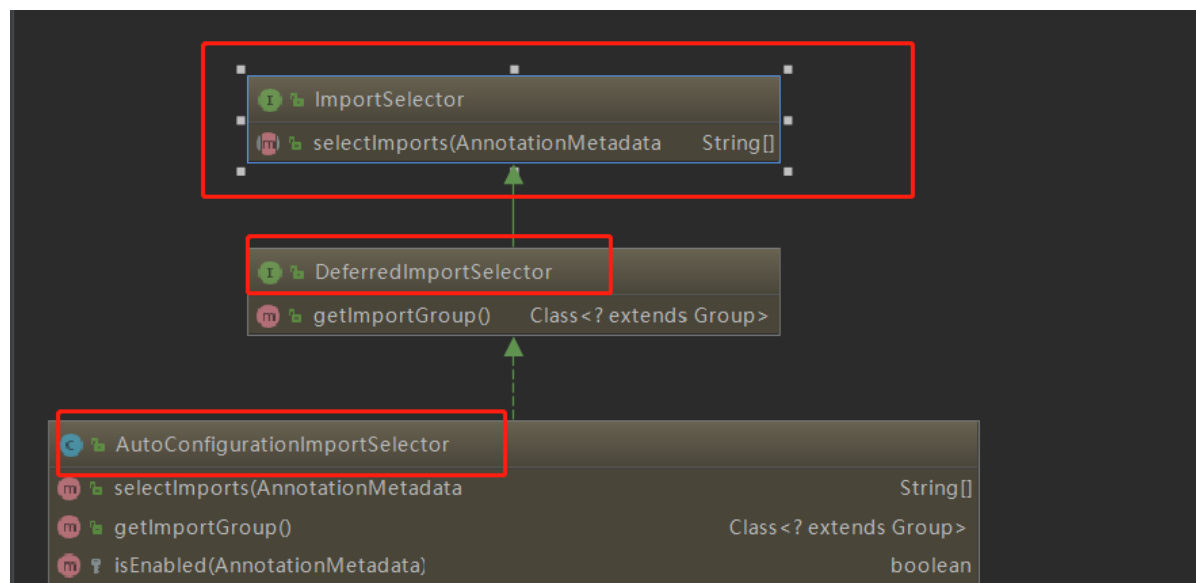
`AutoConfigurationPackage`注解的作用是将添加该注解的类所在的package作为自动配置package进行管理。

可以通过 `AutoConfigurationPackages` 工具类获取自动配置package列表。当通过注解 `@SpringBootApplication` 标注启动类时，已经为启动类添加了 `@AutoConfigurationPackage` 注解。路径为 `@SpringBootApplication -> @EnableAutoConfiguration -> @AutoConfigurationPackage`。也就是说当SpringBoot应用启动时默认会将启动类所在的package作为自动配置的package。

如我们创建了一个sbia-demo的应用，下面包含一个启动模块demo-bootstrap，启动类时Bootstrap，它添加了@SpringBootApplication注解，我们通过测试用例可以看到自动配置package为com.tm.sbia.demo.boot。



1.2.2.2 Import(AutoConfigurationImportSelector.class)



(重点) 可以从图中看出 **AutoConfigurationImportSelector** 实现了 **DeferredImportSelector** 从 **ImportSelector** 继承的方法: **selectImports**。

```
@Override
public String[] selectImports(AnnotationMetadata annotationMetadata) {
    if (!isEnabled(annotationMetadata)) {
        return NO_IMPORTS;
    }
    AutoConfigurationMetadata autoConfigurationMetadata =
        AutoConfigurationMetadataLoader
            .loadMetadata(this.beanClassLoader);
    AnnotationAttributes attributes = getAttributes(annotationMetadata);
    List<String> configurations = getCandidateConfigurations(annotationMetadata,
        attributes);
    configurations = removeDuplicates(configurations);
    Set<String> exclusions = getExclusions(annotationMetadata, attributes);
    checkExcludedClasses(configurations, exclusions);
    configurations.removeAll(exclusions);
    configurations = filter(configurations, autoConfigurationMetadata);
    fireAutoConfigurationImportEvents(configurations, exclusions);
    return StringUtils.toStringArray(configurations);
}
```

第9行List<String> configurations =

getCandidateConfigurations(annotationMetadata, attributes);其实是去加载各个组件jar下的public static final String FACTORIES_RESOURCE_LOCATION = "META-INF/spring.factories";外部文件。

该方法在springboot启动流程——**bean实例化前**被执行，返回要实例化的类信息列表；

如果获取到**类信息**，spring可以通过**类加载器**将**类加载到jvm**中，现在我们已经通过spring-boot的starter依赖方式**依赖了我们需要的组件**，那么这些组件的类信息在select方法中就可以被获取到。

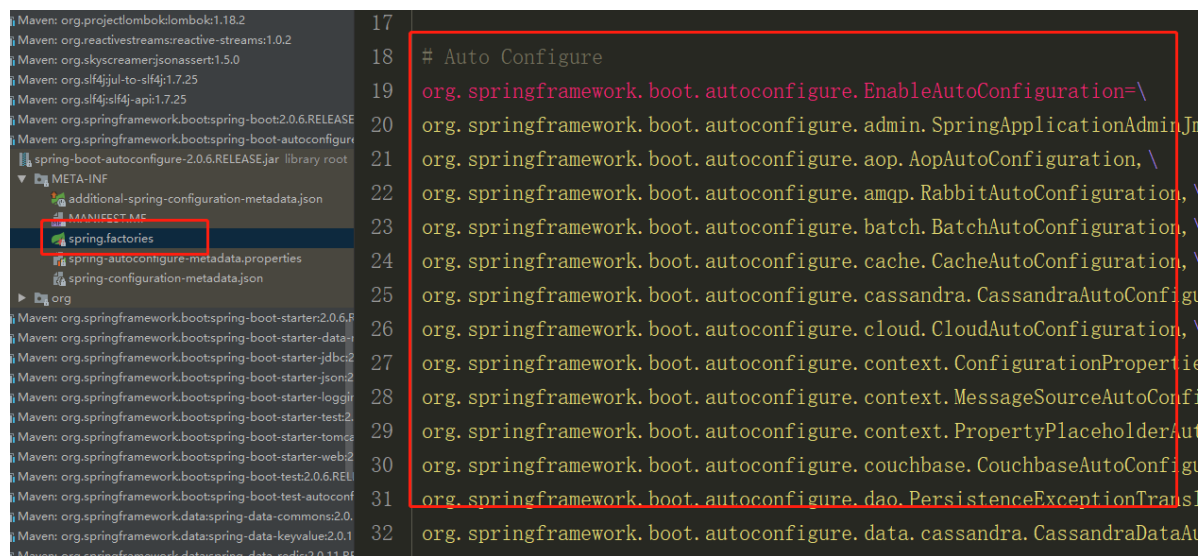
```
protected List<String> getCandidateConfigurations(AnnotationMetadata metadata,
AnnotationAttributes attributes) {
    List<String> configurations =
SpringFactoriesLoader.loadFactoryNames(this.getSpringFactoriesLoaderFactoryClass
(), this.getBeanClassLoader());
    Assert.notEmpty(configurations, "No auto configuration classes found in
META-INF/spring.factories. If you are using a custom packaging, make sure that
file is correct.");
    return configurations;
}
```

其返回一个自动配置类的类名列表，方法调用了loadFactoryNames方法，查看该方法

```
public static List<String> loadFactoryNames(Class<?> factoryClass, @Nullable
ClassLoader classLoader) {
    String factoryClassName = factoryClass.getName();
    return (List)loadSpringFactories(classLoader).getOrDefault(factoryClassName,
Collections.emptyList());
}
```

自动配置器会跟根据传入的factoryClass.getName()到项目系统路径下所有的spring.factories文件中找到相应的key，从而加载里面的类。

这个外部文件，有很多自动配置的类。如下：



(重点) 其中，最关键的要属@Import(AutoConfigurationImportSelector.class)，借助AutoConfigurationImportSelector，@EnableAutoConfiguration可以帮助SpringBoot应用将所有符合条件(spring.factories)的**bean定义**（如java Config@Configuration配置）都加载到当前SpringBoot创建并使用的IoC容器。

1.2.2.3 SpringFactoriesLoader详解

借助于Spring框架**原有**的一个工具类：**SpringFactoriesLoader**的支持，@EnableAutoConfiguration可以智能的自动配置功效才得以大功告成！

SpringFactoriesLoader属于Spring框架私有的一种**扩展**方案，其主要功能就是从**指定的配置文件META-INF/spring.factories**加载配置，**加载工厂类**。

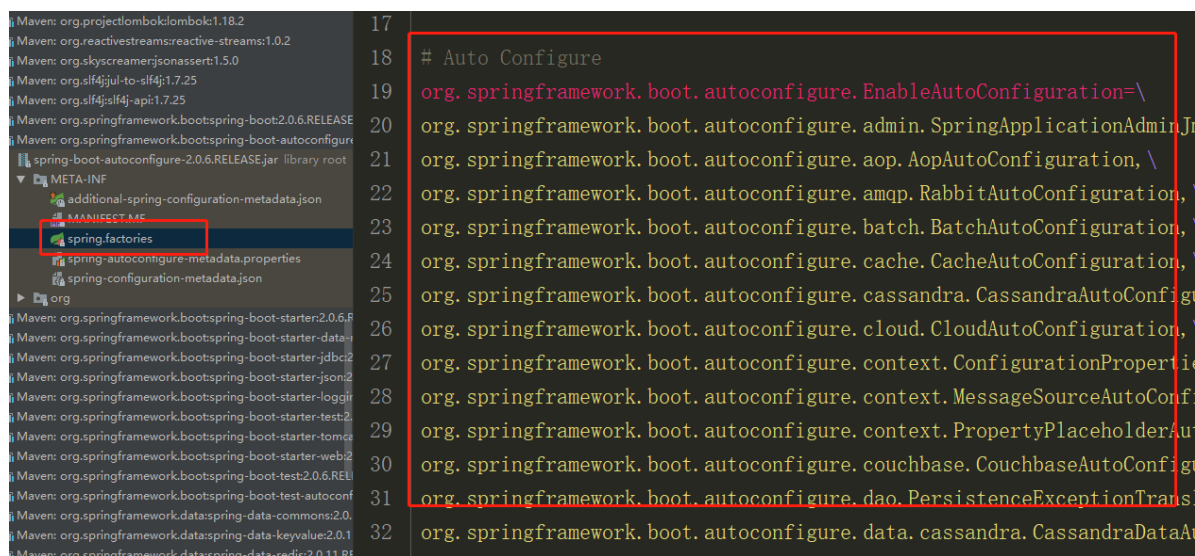
SpringFactoriesLoader为**Spring工厂加载器**，该对象提供了loadFactoryNames方法，入参为factoryClass和classLoader即需要传入**工厂类名称**和**对应的类加载器**，方法会根据指定的classLoader，加载该类加载器搜索路径下的指定文件，即**spring.factories**文件；

传入的**工厂类**为接口，而文件中对应的类则是**接口的实现类**，或最终作为实现类。

```
public abstract class SpringFactoriesLoader {
    //...
    public static <T> List<T> loadFactories(Class<T> factoryClass, ClassLoader
    classLoader) {
        ...
    }

    public static List<String> loadFactoryNames(Class<?> factoryClass,
    ClassLoader classLoader) {
        ....
    }
}
```

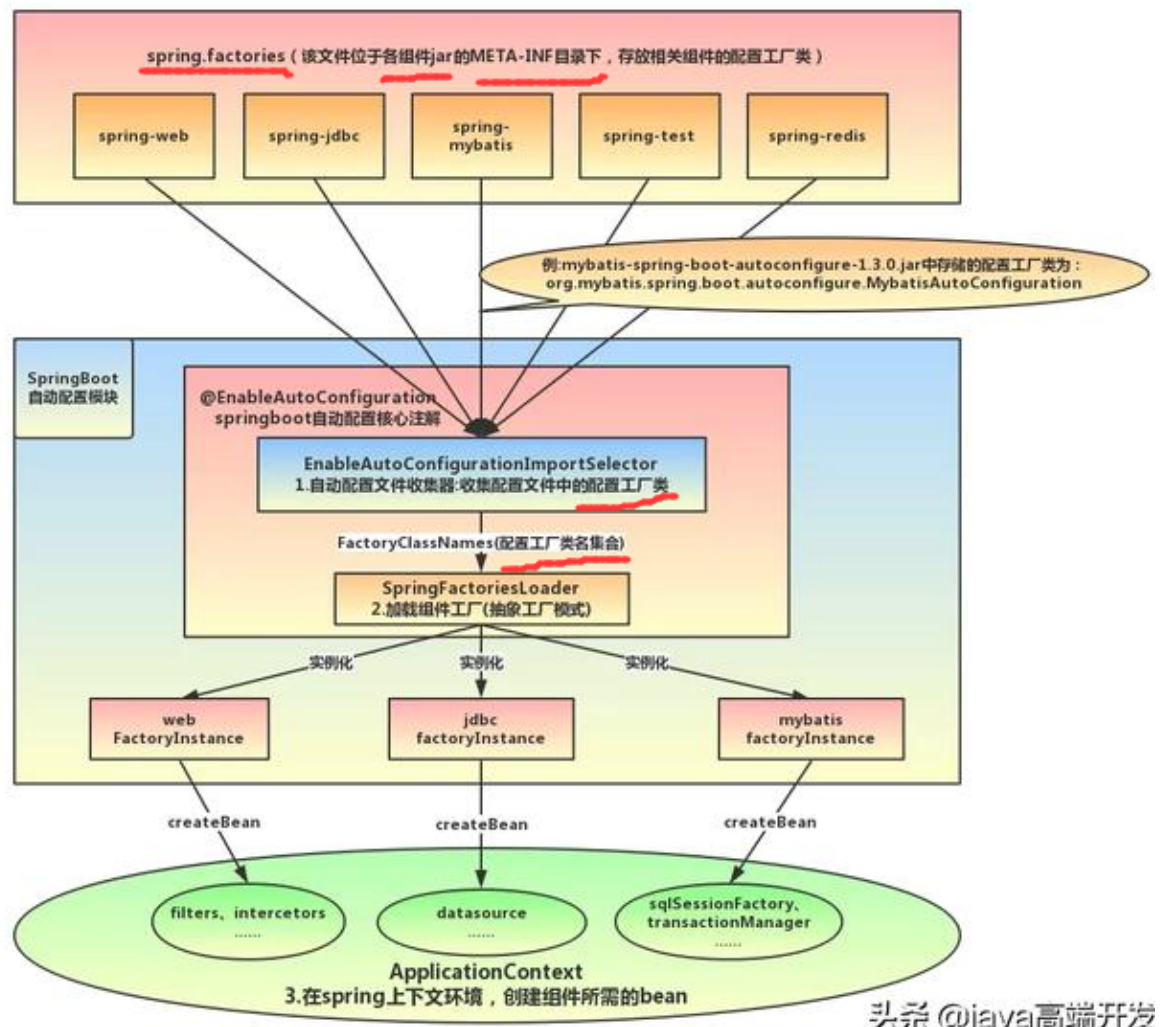
配合@EnableAutoConfiguration使用的话，它更多是提供一种**配置查找**的功能支持，即根据@EnableAutoConfiguration的**完整类名**org.springframework.boot.autoconfigure.EnableAutoConfiguration作为**查找的Key**,获取对应的一组@Configuration类



上图就是从SpringBoot的autoconfigure依赖包中的META-INF/spring.factories配置文件中摘录的一段内容，可以很好地说明问题。

(重点) 所以，@EnableAutoConfiguration自动配置的魔法其实就变成了：

从classpath中搜寻所有的META-INF/spring.factories配置文件，并将其中org.springframework.boot.autoconfigure.EnableAutoConfiguration对应的**配置项**通过**反射 (Java Reflection)** 实例化为对应的标注了@Configuration的JavaConfig形式的IoC容器配置类，然后汇总为一个并加载到IoC容器。



1.3 SpringApplication.run

```
public static void main(String[] args) {
    //代码很简单SpringApplication.run();
    SpringApplication.run(ConsumerApp.class, args);
}
```

```
public static ConfigurableApplicationContext run(Class<?> primarySource,
        String... args) {

    //这个里面调用了run() 方法, 我们转到定义
    return run(new Class<?>[] { primarySource }, args);
}
```

//这个run方法代码也很简单, 就做了两件事情

//1、new了一个SpringApplication() 这么一个对象

//2、执行new出来的SpringApplication()对象的run()方法

```
public static ConfigurableApplicationContext run(Class<?>[] primarySources,
        String[] args) {

    return new SpringApplication(primarySources).run(args);
}
```

上面代码主要做了两件事情。

- 第一步new了一个SpringApplication对象
- 第二步调用了run()方法。

1.3.1 new SpringApplication

```
public SpringApplication(ResourceLoader resourceLoader, Class<?>...
primarySources) {
    this.resourceLoader = resourceLoader;
    Assert.notNull(primarySources, "PrimarySources must not be null");
    //1、先把主类保存起来
    this.primarySources = new LinkedHashSet<>
(Arrays.asList(primarySources));
    //2、判断运行项目的类型
    this.webApplicationType = WebApplicationType.deduceFromClasspath();
    //3、扫描当前路径下META-INF/spring.factories文件的，加载
ApplicationContextInitializer接口实例
    setInitializers((Collection)
getSpringFactoriesInstances(ApplicationContextInitializer.class));
    //4、扫描当前路径下META-INF/spring.factories文件的，加载ApplicationListener接
口实例
    setListeners((Collection)
getSpringFactoriesInstances(ApplicationListener.class));
    this.mainApplicationClass = deduceMainApplicationClass();
}
```

利用SPI机制扫描 META-INF/spring.factories 这个文件，并且加载 **ApplicationContextInitializer**、**ApplicationListener** 接口实例。

- 1、ApplicationContextInitializer 这个类当springboot上下文Context初始化完成后会调用
- 2、ApplicationListener 当springboot启动时事件change后都会触发

下面我们来自定义ApplicationContextInitializer、ApplicationListener 接口实现类

代码见spring-boot-demo项目中的listener

总结：上面就是SpringApplication初始化的代码，new SpringApplication()没做啥事情，利用SPI机制主要加载了META-INF/spring.factories 下面定义的事件监听器接口实现类

1.3.2 执行run()方法

```
public ConfigurableApplicationContext run(String... args) {

    <!--1、这个是一个计时器，没什么好说的-->
    Stopwatch stopwatch = new Stopwatch();
    stopwatch.start();
    DefaultBootstrapContext bootstrapContext = createBootstrapContext();
    ConfigurableApplicationContext context = null;

    <!--2、这个也不是重点，就是设置了一些环境变量-->
    configureHeadlessProperty();

    <!--3、获取事件监听器SpringApplicationRunListener类型，并且执行starting()方法-->
    SpringApplicationRunListeners listeners = getRunListeners(args);
    listeners.starting(bootstrapContext, this.mainApplicationClass);

    try {
```



```

<!--4、把参数args封装成DefaultApplicationArguments, 这个了解一下就知道-->
ApplicationArguments applicationArguments = new
DefaultApplicationArguments(args);

<!--5、这个很重要准备环境了, 并且把环境跟spring上下文绑定好, 并且执行
environmentPrepared()方法-->
//准备容器环境、这里会加载配置文件。在这个方法里面会调用所有监听器Listener的
onApplicationEvent(event);
// 此时有一个与配置文件相关的监听器就会被加载`ConfigFileApplicationListener`
ConfigurableEnvironment environment =
prepareEnvironment(listeners,applicationArguments);

<!--6、判断一些环境的值, 并设置一些环境的值-->
configureIgnoreBeanInfo(environment);

<!--7、打印banner-->
Banner printedBanner = printBanner(environment);

<!--8、创建上下文, 根据项目类型创建上下文-->
context = createApplicationContext();
context.setApplicationStartup(this.applicationStartup);

<!--9、准备上下文, 执行完成后调用contextPrepared()方法,contextLoaded()方法-->
prepareContext(bootstrapContext, context, environment, listeners,
applicationArguments, printedBanner);

<!--10、这个是spring启动的代码了, 这里就回去里面就回去扫描并且初始化单实例bean了-->
//这个refreshContext()加载了bean, 还启动了内置web容器, 需要细细的去看看
refreshContext(context);

<!--11、啥事情都没有做-->
afterRefresh(context, applicationArguments);
stopwatch.stop();
if (this.logStartupInfo) {
    new
StartupInfoLogger(this.mainApplicationClass).logStarted(getApplicationLog(),
stopwatch);
}

<!--12、执行ApplicationRunListeners中的started()方法-->
listeners.started(context);

<!--执行Runner (ApplicationRunner和CommandLineRunner) -->
callRunners(context, applicationArguments);
}
catch (Throwable ex) {
    handleRunFailure(context, ex, listeners);
    throw new IllegalStateException(ex);
}
try {
    listeners.running(context);
}
catch (Throwable ex) {
    handleRunFailure(context, ex, null);
    throw new IllegalStateException(ex);
}
return context;

```

```
}
```

1.3.2.1 环境变量

prepareEnvironment

```
private ConfigurableEnvironment prepareEnvironment(SpringApplicationRunListeners
listeners,
            DefaultBootstrapContext bootstrapContext, ApplicationArguments
applicationArguments) {
    // 创建和配置环境变量
    ConfigurableEnvironment environment = getOrCreateEnvironment();
    configureEnvironment(environment, applicationArguments.getSourceArgs());
    ConfigurationPropertySources.attach(environment);
    listeners.environmentPrepared(bootstrapContext, environment);
    DefaultPropertiesPropertySource.moveToEnd(environment);
    Assert.state(!environment.containsProperty("spring.main.environment-
prefix"),
                "Environment prefix cannot be set via properties.");
    bindToSpringApplication(environment);
    if (!this.isCustomEnvironment) {
        environment = new
EnvironmentConverter(getClassLoader()).convertEnvironmentIfNecessary(environment
,
                deduceEnvironmentClass());
    }
    ConfigurationPropertySources.attach(environment);
    return environment;
}
```

getOrCreateEnvironment()

```
/**
 * 该方法根据webApplicationType判断当前项目是什么类型项目
 */
private ConfigurableEnvironment getOrCreateEnvironment() {
    if (this.environment != null) {
        return this.environment;
    }
    switch (this.webApplicationType) {
        case SERVLET:
            return new ApplicationServletEnvironment();
        case REACTIVE:
            return new ApplicationReactiveWebEnvironment();
        default:
            return new ApplicationEnvironment();
    }
}

//webApplicationType是在new SpringApplication方法中通过
webApplicationType.deduceFromClasspath()进行赋值
```

应用类型

枚举WebApplicationType中定义了三个应用类型：

- NONE：应用程序不作为web应用启动，不启动内嵌的服务。
- SERVLET：应用程序以基于servlet的web应用启动，需启动内嵌servlet web服务。
- REACTIVE：应用程序以响应式web应用启动，需启动内嵌的响应式web服务。

推断应用类型

SpringBoot启动时，在创建SpringApplication的构造方法内会调用枚举WebApplicationType的deduceFromClasspath方法获得应用类型并设置当前应用是普通web应用、响应式web应用还是非web应用。

SpringApplication的构造方法中调用并设置源代码：

```
this.webApplicationType = webApplicationType.deduceFromClasspath();
```

deduceFromClasspath方法由枚举WebApplicationType提供，具体实现如下：

```
static webApplicationType deduceFromClasspath() {
    if (ClassUtils.isPresent(WEBFLUX_INDICATOR_CLASS, null)
        && !ClassUtils.isPresent(WEBMVC_INDICATOR_CLASS, null)
        && !ClassUtils.isPresent(JERSEY_INDICATOR_CLASS, null)) {
        return webApplicationType.REACTIVE;
    }
    for (String className : SERVLET_INDICATOR_CLASSES) {
        if (!ClassUtils.isPresent(className, null)) {
            return webApplicationType.NONE;
        }
    }
    return webApplicationType.SERVLET;
}
```

推断的过程中重点调用了ClassUtils.isPresent()方法，用来判断指定类名的类是否存在，是否可以进行加载。ClassUtils.isPresent()方法源代码如下：

```
public static boolean isPresent(String className, @Nullable ClassLoader
classLoader) {
    try {
        forName(className, classLoader);
        return true;
    } catch (IllegalAccessException err) {
        throw new IllegalStateException("Readability mismatch in inheritance
hierarchy of class [" +
            className + "]: " + err.getMessage(), err);
    } catch (Throwable ex) {
        return false;
    }
}
```

isPresent()方法调用了forName()方法，如果在调用forName()方法的过程中出现异常则返回false，也就是目标类不存在。否则，返回true。

看一下forName()方法的部分代码：

```

public static Class<?> forName(String name, @Nullable ClassLoader classLoader)
    throws ClassNotFoundException, LinkageError {

    // 此处省略一些非空和基础类型的判断逻辑代码

    ClassLoader clToUse = classLoader;
    if (clToUse == null) {
        //如果为空则获取默认ClassLoader
        clToUse = getDefaultClassLoader();
    }
    try {
        // 返回加载类的Class。
        return Class.forName(name, false, clToUse);
    } catch (ClassNotFoundException ex) {
        // 如果直接加载类出现异常，则尝试加载内部类。
        int lastDotIndex = name.lastIndexOf(PACKAGE_SEPARATOR);
        if (lastDotIndex != -1) {
            // 拼接内部类
            String innerClassName =
                name.substring(0, lastDotIndex) + INNER_CLASS_SEPARATOR +
                name.substring(lastDotIndex + 1);
            try {
                return Class.forName(innerClassName, false, clToUse);
            }
            catch (ClassNotFoundException ex2) {
                // Swallow - let original exception get through
            }
        }
        throw ex;
    }
}

```

通过以上核心代码，可得知forName()方法主要做的事情就是获得类加载器，尝试直接加载类，如果失败则尝试加载该类的内部类，如果依旧失败，则抛出异常。

因此，整个应用类型的推断分以下步骤：

- SpringBoot调用SpringApplication构造方法；
- SpringApplication构造方法调用枚举类的类型推断方法deduceFromClasspath()。
- deduceFromClasspath()方法通过ClassUtils.isPresent()返回结果为true或false来确定是否加载成功指定的类。
- ClassUtils.isPresent()方法通过调用forName()方法并捕获异常来确定是否能够成功加载该类。
- forName()方法通过尝试加载指定类和指定类的内部类来确定该类是否存在，存在则返回该类，不存在则抛异常。

在类型推断的过程中枚举类WebApplicationType定义了具体去加载哪些类：

```

private static final String[] SERVLET_INDICATOR_CLASSES = {
    "javax.servlet.Servlet",
    "org.springframework.web.context.ConfigurableWebApplicationContext"
};

private static final String WEBMVC_INDICATOR_CLASS = "org.springframework."
    + "web.servlet.DispatcherServlet";

private static final String WEBFLUX_INDICATOR_CLASS = "org."
    + "springframework.web.reactive.DispatcherHandler";

private static final String JERSEY_INDICATOR_CLASS =
    "org.glassfish.jersey.servlet.ServletContainer";

```

- 如果应用程序存在DispatcherHandler并且不存在DispatcherServlet和ServletContainer则为响应式web应用，需加载并启动内嵌的响应式web服务。
- 如果应用程序不包含Servlet和ConfigurableWebApplicationContext则为普通应用程序。
- 其他情况则为基于servlet的web应用，需加载并启动内嵌的web服务。

通过上面webApplicationType，返回到getOrCreateEnvironment方法，调用new ApplicationServletEnvironment();方法；

ApplicationServletEnvironment继承了StandardServletEnvironment，StandardServletEnvironment又继承了StandardEnvironment方法，StandardEnvironment又继承了AbstractEnvironment方法；在AbstractEnvironment方法中调用了customizePropertySources方法

```

protected AbstractEnvironment(MutablePropertySources propertySources) {
    this.propertySources = propertySources;
    this.propertyResolver = createPropertyResolver(propertySources);
    customizePropertySources(propertySources);
}

```

customizePropertySources方法会回调StandardServletEnvironment方法中的customizePropertySources方法，

```

protected void customizePropertySources(MutablePropertySources propertySources)
{
    propertySources.addLast(new
    StubPropertySource(SERVLET_CONFIG_PROPERTY_SOURCE_NAME));
    propertySources.addLast(new
    StubPropertySource(SERVLET_CONTEXT_PROPERTY_SOURCE_NAME));
    if (JndiLocatorDelegate.isDefaultJndiEnvironmentAvailable()) {
        propertySources.addLast(new
        JndiPropertySource(JNDI_PROPERTY_SOURCE_NAME));
    }
    //在这里又调用了父类StandardEnvironment的方法
    super.customizePropertySources(propertySources);
}

```

到这里为止propertySources里面就加载了servletConfigInitParams、servletContextInitParams、systemProperties、systemEnvironment

然后回到prepareEnvironment方法中，在listeners.environmentPrepared(bootstrapContext, environment);方法中去进行监听

environmentPrepared

```
void environmentPrepared(ConfigurableBootstrapContext bootstrapContext,
    ConfigurableEnvironment environment) {
    doWithListeners("spring.boot.application.environment-prepared",
        (listener) -> listener.environmentPrepared(bootstrapContext,
            environment));
}
```

继续进入environmentPrepared方法，会进入到SpringApplicationRunListener接口，这个接口在run方法中的getRunListeners里面获取，最终是在spring.factories里面进行加载实现类EventPublishingRunListener，执行的是EventPublishingRunListener类中的environmentPrepared方法。

```
public void environmentPrepared(ConfigurableBootstrapContext bootstrapContext,
    ConfigurableEnvironment environment) {
    this.initialMulticaster.multicastEvent(
        new ApplicationEnvironmentPreparedEvent(bootstrapContext,
            this.application, this.args, environment));
}
```

multicastEvent

```
public void multicastEvent(final ApplicationEvent event, @Nullable
    ResolvableType eventType) {
    ResolvableType type = (eventType != null ? eventType :
        resolveDefaultEventType(event));
    Executor executor = getTaskExecutor();
    for (ApplicationListener<?> listener : getApplicationListeners(event, type))
    {
        if (executor != null) {
            executor.execute(() -> invokeListener(listener, event));
        }
        else {
            invokeListener(listener, event);
        }
    }
}
```

invokeListener

```
protected void invokeListener(ApplicationListener<?> listener, ApplicationEvent
    event) {
    ErrorHandler errorHandler = getErrorHandler();
    if (errorHandler != null) {
        try {
            doInvokeListener(listener, event);
        }
        catch (Throwable err) {
            errorHandler.handleError(err);
        }
    }
}
```



```

    }
}
else {
    doInvokeListener(listener, event);
}
}
}

```

doInvokeListener

```

private void doInvokeListener(ApplicationListener listener, ApplicationEvent
event) {
    try {
        listener.onApplicationEvent(event);
    }
    catch (ClassCastException ex) {
        String msg = ex.getMessage();
        if (msg == null || matchesClassCastMessage(msg, event.getClass()) ||
            (event instanceof PayloadApplicationEvent &&
             matchesClassCastMessage(msg, ((PayloadApplicationEvent)
event).getPayload().getClass()))) {
            // Possibly a lambda-defined listener which we could not resolve the
generic event type for
            // -> let's suppress the exception.
            Log loggerToUse = this.lazyLogger;
            if (loggerToUse == null) {
                loggerToUse = LogFactory.getLog(getClass());
                this.lazyLogger = loggerToUse;
            }
            if (loggerToUse.isTraceEnabled()) {
                loggerToUse.trace("Non-matching event type for listener: " +
listener, ex);
            }
        }
        else {
            throw ex;
        }
    }
}
}

```

进入ConfigFileApplicationListener实现类中的onApplicationEvent方法

onApplicationEvent

```

public void onApplicationEvent(ApplicationEvent event) {
    if (event instanceof ApplicationEnvironmentPreparedEvent) {
        //在这个方法里面读取配置文件

        onApplicationEnvironmentPreparedEvent((ApplicationEnvironmentPreparedEvent)
event);
    }
    if (event instanceof ApplicationPreparedEvent) {
        onApplicationPreparedEvent(event);
    }
}
}

```

```

private void
onApplicationEnvironmentPreparedEvent(ApplicationEnvironmentPreparedEvent event)
{
    List<EnvironmentPostProcessor> postProcessors = loadPostProcessors();
    postProcessors.add(this);
    AnnotationAwareOrderComparator.sort(postProcessors);
    for (EnvironmentPostProcessor postProcessor : postProcessors) {
        //进入
        postProcessor.postProcessEnvironment(event.getEnvironment(),
event.getSpringApplication());
    }
}

```

```

public void postProcessEnvironment(ConfigurableEnvironment environment,
SpringApplication application) {
    //进入
    addPropertySources(environment, application.getResourceLoader());
}

```

```

protected void addPropertySources(ConfigurableEnvironment environment,
ResourceLoader resourceLoader) {
    RandomValuePropertySource.addToEnvironment(environment);
    //load方法是读取配置文件的核心方法
    new Loader(environment, resourceLoader).load();
}

```

```

void load() {
    FilteredPropertySource.apply(this.environment, DEFAULT_PROPERTIES,
LOAD_FILTERED_PROPERTY,
                                (defaultProperties) -> {
                                    this.profiles = new LinkedList<>();
                                    this.processedProfiles = new LinkedList<>();

                                    this.activatedProfiles = false;
                                    this.loaded = new LinkedHashMap<>();
                                    initializeProfiles();
                                    while (!this.profiles.isEmpty()) {
                                        Profile profile = this.profiles.poll();
                                        if (isDefaultProfile(profile)) {

addProfileToEnvironment(profile.getName());
                                        }
                                        load(profile,
this::getPositiveProfileFilter,

                                        addToLoaded(MutablePropertySources::addLast, false));
                                        this.processedProfiles.add(profile);
                                    }
                                    load(null, this::getNegativeProfileFilter,
addToLoaded(MutablePropertySources::addFirst, true));
                                    addLoadedPropertySources();
                                    applyActiveProfiles(defaultProperties);
                                });
}

```

1.3.2.2 创建上下文

createApplicationContext()

一起来看下context = createApplicationContext(); 这段代码，这段代码主要是根据项目类型创建上下文，并且会注入几个核心组件类。

```
protected ConfigurableApplicationContext createApplicationContext() {
    Class<?> contextClass = this.applicationContextClass;
    if (contextClass == null) {
        try {
            switch (this.webApplicationType) {
                case SERVLET:
                    contextClass = Class.forName(DEFAULT_SERVLET_WEB_CONTEXT_CLASS);
                    break;
                case REACTIVE:
                    contextClass =
                        Class.forName(DEFAULT_REACTIVE_WEB_CONTEXT_CLASS);
                    break;
                default:
                    contextClass = Class.forName(DEFAULT_CONTEXT_CLASS);
            }
        }
        catch (ClassNotFoundException ex) {
            throw new IllegalStateException(
                "Unable to create a default ApplicationContext, please specify
                an ApplicationContextClass", ex);
        }
    }
    return (ConfigurableApplicationContext)
        BeanUtils.instantiateClass(contextClass);
}
```

```
public
AnnotationConfigServletWebServerApplicationContext(DefaultListableBeanFactory
beanFactory) {
    super(beanFactory);
    //1: 会去注入一些spring核心组件
    this.reader = new AnnotatedBeanDefinitionReader(this);
    this.scanner = new ClassPathBeanDefinitionScanner(this);
}
```

Web类型项目创建上下文对象 AnnotationConfigServletWebServerApplicationContext。这里会把 ConfigurationClassPostProcessor、AutowiredAnnotationBeanPostProcessor 等一些核心组件加入到Spring容器

1.3.2.3 刷新上下文

下面一起来看下refreshContext(context)这个方法，这个方法启动spring的代码加载了bean，还启动了内置web容器

```

private void refreshContext(ConfigurableApplicationContext context) {
    // 转到定义看看
    refresh(context);
    if (this.registerShutdownHook) {
        try {
            context.registerShutdownHook();
        }
        catch (AccessControlException ex) {
            // Not allowed in some environments.
        }
    }
}
}

```

```

protected void refresh(ApplicationContext applicationContext) {
    Assert.isInstanceOf(AbstractApplicationContext.class,
        applicationContext);
    //看看refresh()方法去
    ((AbstractApplicationContext) applicationContext).refresh();
}

```

转到AbstractApplicationContext ->refresh()方法里面发现这是spring容器启动代码

```

/**
 * 加载或刷新一个持久化的配置，可能是XML文件、属性文件或关系数据库模式。
 * 由于这是一种启动方法，如果失败，应该销毁已经创建的单例，以避免悬空资源。
 * 换句话说，在调用该方法之后，要么全部实例化，要么完全不实例化。
 * @throws BeansException 如果bean工厂无法初始化，则抛出 BeansException 异常
 * @throws IllegalStateException 如果已经初始化且不支持多次刷新，则会抛出 IllegalStateException 异常
 */
@Override
public void refresh() throws BeansException, IllegalStateException {
    //加载或刷新配置前的同步处理
    synchronized (this.startupShutdownMonitor) {
        // 为刷新而准备此上下文
        prepareRefresh();

        // 告诉子类去刷新内部bean工厂。
        ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();

        // 准备好bean工厂，以便在此上下文中使用。
        prepareBeanFactory(beanFactory);

        try {
            // 允许在上下文子类中对bean工厂进行后置处理。
            postProcessBeanFactory(beanFactory);

            // 调用在上下文中注册为bean的工厂处理器。
            invokeBeanFactoryPostProcessors(beanFactory);

            // 注册拦截bean创建的bean处理器。
            registerBeanPostProcessors(beanFactory);

            // 初始化此上下文的 message resource 消息资源。
            initMessageSource();

```

```

        // 为这个上下文初始化事件多路广播器。
        initApplicationEventMulticaster();

        // 初始化特定上下文子类中的其他特殊bean。
        onRefresh();

        // 注册监听器（检查监听器的bean并注册它们）。
        registerListeners();

        // 实例化所有剩余的(非 lazy-init 懒初始化的)单例。
        finishBeanFactoryInitialization(beanFactory);

        // 最后一步：发布相应的事件。
        finishRefresh();
    }

    catch (BeansException ex) {
        if (logger.isWarnEnabled()) {
            logger.warn("Exception encountered during context initialization
- " +
                        "cancelling refresh attempt: " + ex);
        }

        // 销毁已经创建的单例，以避免悬空资源。
        destroyBeans();

        // 重置 'active' 表示。
        cancelRefresh(ex);

        // 将异常传播给调用者。
        throw ex;
    }

    finally {
        // 重置Spring内核中的共用的缓存，因为我们可能再也不需要单例bean的元数据了.....
        resetCommonCaches();
    }
}
}
}

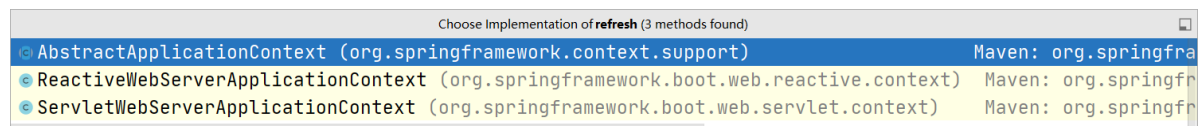
```

spring容器启动代码就不说了，这里主要看一下onRefresh()这个方法。转到定义发现这个方法里面啥都没有，这明显是一个钩子方法，它会钩到它子类重写onRefresh()方法。所以去看子类里面的onRefresh()

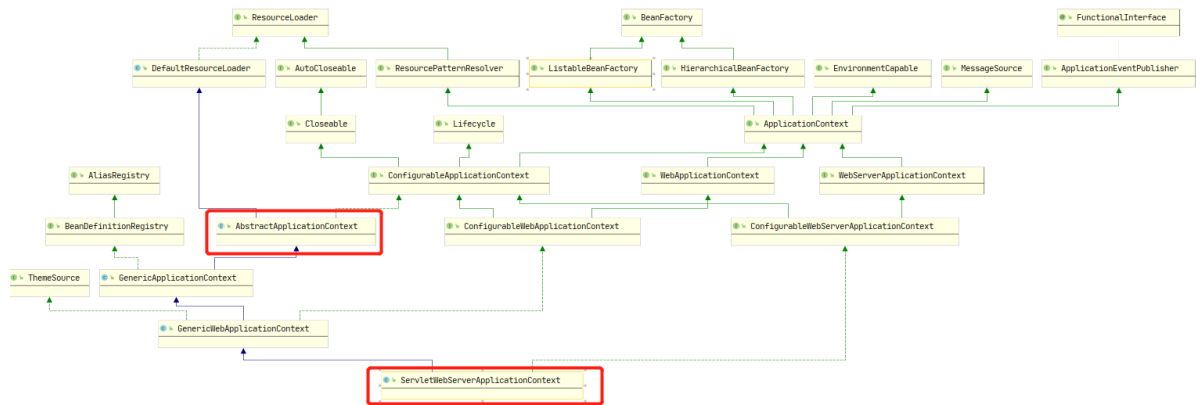
```

protected void onRefresh() throws BeansException {
    //这是一个空方法，AbstractApplicationContext 这个类是一个抽象类，
    //所以我们要找到集成AbstractApplicationContext的子类，去看子类里面的onRefresh()
    // For subclasses: do nothing by default.
}

```



我们这里是一个Web项目，所以我们就去看 ServletWebServerApplicationContext 这个类，我还是把类的关系图贴一下



我们就去看 ServletWebServerApplicationContext 这个类下面的 onRefresh() 方法

```

protected void onRefresh() {
    super.onRefresh();
    try {
        //看到内置容器的影子了，进去看看
        createWebServer();
    }
    catch (Throwable ex) {
        throw new ApplicationContextException("Unable to start web server",
ex);
    }
}

```

```

private void createWebServer() {
    webServer webServer = this.webServer;
    ServletContext servletContext = getServletContext();
    if (webServer == null && servletContext == null) {
        //1、这个获取webServerFactory还是要进去看看
        ServletWebServerFactory factory = getWebServerFactory();
        this.webServer = factory.getWebServer(getSelfInitializer());
    }
    else if (servletContext != null) {
        try {
            getSelfInitializer().onStartup(servletContext);
        }
        catch (ServletException ex) {
            throw new ApplicationContextException("Cannot initialize servlet
context",
                ex);
        }
    }
    initPropertySources();
}

```

我们继续看下getWebServletFactory() 这个方法，这个里面其实就是选择出哪种类型的web容器了

```

protected ServletWebServerFactory getWebServerFactory() {
    // Use bean names so that we don't consider the hierarchy
    String[] beanNames = getBeanFactory()

```

```

        .getBeanNamesForType(ServletWebServerFactory.class);
        if (beanNames.length == 0) {
            throw new ApplicationContextException(
                "Unable to start ServletWebServerApplicationContext due to
missing "
                    + "ServletWebServerFactory bean.");
        }
        if (beanNames.length > 1) {
            throw new ApplicationContextException(
                "Unable to start ServletWebServerApplicationContext due to
multiple "
                    + "ServletWebServerFactory beans : "
                    +
                StringUtils.arrayToCommaDelimitedString(beanNames));
        }
        return getBeanFactory().getBean(beanNames[0],
            ServletWebServerFactory.class);
    }

```

我们再回头去看factory.getWebServer(getSelfInitializer()), 转到定义就会看到很熟悉的名字tomcat

```

public WebServer getWebServer(ServletContextInitializer... initializers) {
    //tomcat这位大哥出现了
    Tomcat tomcat = new Tomcat();
    File baseDir = (this.baseDirectory != null ? this.baseDirectory
        : createTempDir("tomcat"));
    tomcat.setBaseDir(baseDir.getAbsolutePath());
    Connector connector = new Connector(this.protocol);
    tomcat.getService().addConnector(connector);
    customizeConnector(connector);
    tomcat.setConnector(connector);
    tomcat.getHost().setAutoDeploy(false);
    configureEngine(tomcat.getEngine());
    for (Connector additionalConnector : this.additionalTomcatConnectors) {
        tomcat.getService().addConnector(additionalConnector);
    }
    prepareContext(tomcat.getHost(), initializers);
    return getTomcatWebServer(tomcat);
}

```

内置的Servlet容器就是在onRefresh() 方法里面启动的，至此一个Servlet容器就启动OK了。

1.4 总结

- 1、new了一个SpringApplication对象，使用SPI技术加载加载 ApplicationContextInitializer、ApplicationListener 接口实例
- 2、调用SpringApplication.run() 方法
- 3、调用createApplicationContext()方法创建上下文对象，创建上下文对象同时会注册spring的核心组件类（ConfigurationClassPostProcessor、AutowiredAnnotationBeanPostProcessor 等）。
- 4、调用refreshContext() 方法启动Spring容器和内置的Servlet容器

2 SPI

SPI，全称为 Service Provider Interface，是一种服务发现机制。它通过在ClassPath路径下的META-INF/services文件夹查找文件，自动加载文件里所定义的类。

这一机制为很多框架扩展提供了可能，比如在Dubbo、JDBC中都使用到了SPI机制。我们先通过一个简单的例子来看下它是怎么用的。

2.1 例子

首先，我们需要定义一个接口，SPIService

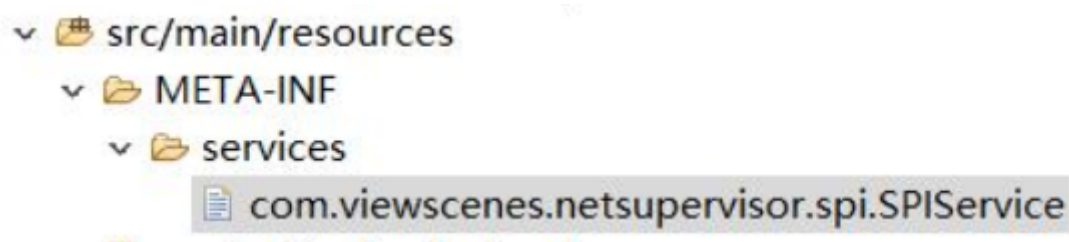
```
package com.viewscenes.netsupervisor.spi;  
public interface SPIService {  
    void execute();  
}
```

然后，定义两个实现类，没别的意思，只输入一句话。

```
package com.viewscenes.netsupervisor.spi;  
public class SpiImpl1 implements SPIService{  
    public void execute() {  
        System.out.println("SpiImpl1.execute()");  
    }  
}  
-----我是乖巧的分割线-----  
package com.viewscenes.netsupervisor.spi;  
public class SpiImpl2 implements SPIService{  
    public void execute() {  
        System.out.println("SpiImpl2.execute()");  
    }  
}
```

最后呢，要在ClassPath路径下配置添加一个文件。文件名字是接口的全限定类名，内容是实现类的全限定类名，多个实现类用换行符分隔。

文件路径如下：



内容就是实现类的全限定类名：

```
com.viewscenes.netsupervisor.spi.SpiImpl1  
com.viewscenes.netsupervisor.spi.SpiImpl2
```

2.2 测试

然后我们就可以通过 `ServiceLoader.load` 或者 `Service.providers` 方法拿到实现类的实例。其中，`Service.providers` 包位于 `sun.misc.Service`，而 `ServiceLoader.load` 包位于 `java.util.ServiceLoader`。

```

public class Test {
    public static void main(String[] args) {
        Iterator<SPIService> providers = Service.providers(SPIService.class);
        ServiceLoader<SPIService> load = ServiceLoader.load(SPIService.class);

        while(providers.hasNext()) {
            SPIService ser = providers.next();
            ser.execute();
        }
        System.out.println("-----");
        Iterator<SPIService> iterator = load.iterator();
        while(iterator.hasNext()) {
            SPIService ser = iterator.next();
            ser.execute();
        }
    }
}

```

两种方式的输出结果是一致的：

```

SpiImpl1.execute()
SpiImpl2.execute()
-----
SpiImpl1.execute()
SpiImpl2.execute()

```

2.3 源码分析

我们看到一个位于 `sun.misc` 包，一个位于 `java.util` 包，`sun` 包下的源码看不到。我们就以 `ServiceLoader.load` 为例，通过源码看看它里面到底怎么做的。

2.3.1 ServiceLoader

首先，我们先了解下 `ServiceLoader`，看看它的类结构。

```

public final class ServiceLoader<S> implements Iterable<S>
    //配置文件的路径
    private static final String PREFIX = "META-INF/services/";
    //加载的服务类或接口
    private final Class<S> service;
    //已加载的服务类集合
    private LinkedHashMap<String,S> providers = new LinkedHashMap<>();
    //类加载器
    private final ClassLoader loader;
    //内部类，真正加载服务类
    private LazyIterator lookupIterator;
}

```

2.3.2 Load

`load` 方法创建了一些属性，重要的是实例化了内部类，`LazyIterator`。最后返回 `ServiceLoader` 的实例。

```

public final class ServiceLoader<S> implements Iterable<S>
    private ServiceLoader(Class<S> svc, ClassLoader cl) {
        //要加载的接口
    }

```

```

        service = Objects.requireNonNull(svc, "Service interface cannot be
null");
        //类加载器
        loader = (cl == null) ? ClassLoader.getSystemClassLoader() : cl;
        //访问控制器
        acc = (System.getSecurityManager() != null) ?
AccessController.getContext() : null;
        //先清空
        providers.clear();
        //实例化内部类
        LazyIterator lookupIterator = new LazyIterator(service, loader);
    }
}

```

2.3.3 查找实现类

查找实现类和创建实现类的过程，都在LazyIterator完成。当我们调用iterator.hasNext和iterator.next方法的时候，实际上调用的都是LazyIterator的相应方法。

```

public Iterator<S> iterator() {
    return new Iterator<S>() {
        public boolean hasNext() {
            return lookupIterator.hasNext();
        }
        public S next() {
            return lookupIterator.next();
        }
        .....
    };
}

```

所以，我们重点关注lookupIterator.hasNext()方法，它最终会调用到hasNextService。

```

private class LazyIterator implements Iterator<S>{
    Class<S> service;
    ClassLoader loader;
    Enumeration<URL> configs = null;
    Iterator<String> pending = null;
    String nextName = null;
    private boolean hasNextService() {
        //第二次调用的时候，已经解析完成了，直接返回
        if (nextName != null) {
            return true;
        }
        if (configs == null) {
            //META-INF/services/ 加上接口的全限定类名，就是文件服务类的文件
            //META-INF/services/com.viewscenes.net.supervisor.spi.SPIService
            String fullName = PREFIX + service.getName();
            //将文件路径转成URL对象
            configs = loader.getResources(fullName);
        }
        while ((pending == null) || !pending.hasNext()) {
            //解析URL文件对象，读取内容，最后返回
            pending = parse(service, configs.nextElement());
        }
        //拿到第一个实现类的类名
    }
}

```

```

        nextName = pending.next();
        return true;
    }
}

```

2.3.4 创建实例

当然，调用next方法的时候，实际调用到的是，lookupIterator.nextService。它通过反射的方式，创建实现类的实例并返回。

```

private class LazyIterator implements Iterator<S>{
    private S nextService() {
        //全限定类名
        String cn = nextName;
        nextName = null;
        //创建类的Class对象
        Class<?> c = Class.forName(cn, false, loader);
        //通过newInstance实例化
        S p = service.cast(c.newInstance());
        //放入集合，返回实例
        providers.put(cn, p);
        return p;
    }
}

```

看到这儿，我想已经很清楚了。获取到类的实例，我们自然就可以对它为所欲为了！

2.4 JDBC中的应用

我们开头说，SPI机制为很多框架的扩展提供了可能，其实JDBC就应用到了这一机制。回忆一下JDBC获取数据库连接的过程。在早期版本中，需要先设置数据库驱动的连接，再通过DriverManager.getConnection获取一个Connection。

```

String url = "jdbc:mysql:///consult?serverTimezone=UTC";
String user = "root";
String password = "root";

Class.forName("com.mysql.jdbc.Driver");
Connection connection = DriverManager.getConnection(url, user, password);

```

在较新版本中(具体哪个版本，笔者没有验证)，设置数据库驱动连接，这一步骤就不再需要，那么它是如何分辨是哪种数据库的呢？答案就在SPI。

2.4.1 加载

我们把目光回到DriverManager类，它在静态代码块里面做了一件比较重要的事。很明显，它已经通过SPI机制，把数据库驱动连接初始化了。

```

public class DriverManager {
    static {
        loadInitialDrivers();
        println("JDBC DriverManager initialized");
    }
}

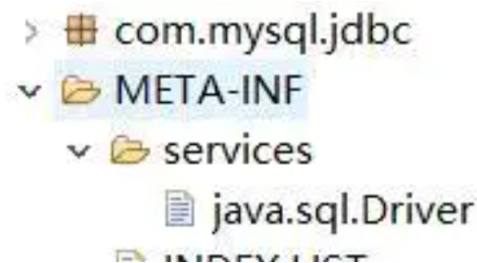
```

具体过程还得看loadInitialDrivers，它在里面查找的是Driver接口的服务类，所以它的文件路径就是：META-INF/services/java.sql.Driver。

```
public class DriverManager {
    private static void loadInitialDrivers() {
        AccessController.doPrivileged(new PrivilegedAction<Void>() {
            public void run() {
                //很明显，它要加载Driver接口的服务类，Driver接口的包为:java.sql.Driver
                //所以它要找的就是META-INF/services/java.sql.Driver文件
                ServiceLoader<Driver> loadedDrivers =
                ServiceLoader.load(Driver.class);
                Iterator<Driver> driversIterator = loadedDrivers.iterator();
                try{
                    //查到之后创建对象
                    while(driversIterator.hasNext()) {
                        driversIterator.next();
                    }
                } catch(Throwable t) {
                    // Do nothing
                }
                return null;
            }
        });
    }
}
```

那么，这个文件哪里呢？我们来看MySQL的jar包，就是这个文件，文件内容为：

com.mysql.cj.jdbc.Driver



2.4.2 创建实例

上一步已经找到了MySQL中的com.mysql.cj.jdbc.Driver全限定类名，当调用next方法时，就会创建这个类的实例。它就完成了一件事，向DriverManager注册自身的实例。

```
public class Driver extends NonRegisteringDriver implements java.sql.Driver {
    static {
        try {
            //注册
            //调用DriverManager类的注册方法
            //往registeredDrivers集合中加入实例
            java.sql.DriverManager.registerDriver(new Driver());
        } catch (SQLException E) {
            throw new RuntimeException("Can't register driver!");
        }
    }
    public Driver() throws SQLException {
        // Required for Class.forName().newInstance()
    }
}
```

```
}
```

2.4.3 创建Connection

在DriverManager.getConnection()方法就是创建连接的地方，它通过循环已注册的数据库驱动程序，调用其connect方法，获取连接并返回。

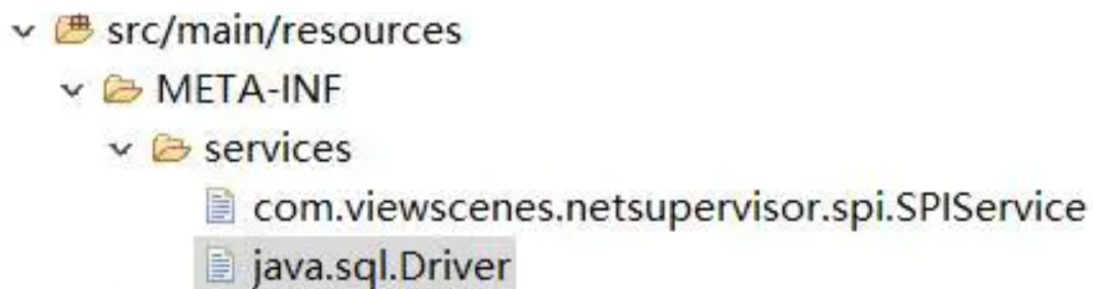
```
private static Connection getConnection(  
    String url, java.util.Properties info, Class<?> caller) throws  
SQLException {  
    //registeredDrivers中就包含com.mysql.cj.jdbc.Driver实例  
    for(DriverInfo aDriver : registeredDrivers) {  
        if(isDriverAllowed(aDriver.driver, callerCL)) {  
            try {  
                //调用connect方法创建连接  
                Connection con = aDriver.driver.connect(url, info);  
                if (con != null) {  
                    return (con);  
                }  
            } catch (SQLException ex) {  
                if (reason == null) {  
                    reason = ex;  
                }  
            }  
        } else {  
            println("    skipping: " + aDriver.getClass().getName());  
        }  
    }  
}
```

2.4.4 再扩展

既然我们知道JDBC是这样创建数据库连接的，我们能不能再扩展一下呢？如果我们自己也创建一个java.sql.Driver文件，自定义实现类MyDriver，那么，在获取连接的前后就可以动态修改一些信息。

还是先在项目ClassPath下创建文件，文件内容为自定义驱动类

```
com.viewscenes.netsupervisor.spi.MyDriver
```



我们的MyDriver实现类，继承自MySQL中的NonRegisteringDriver，还要实现java.sql.Driver接口。这样，在调用connect方法的时候，就会调用到此类，但实际创建的过程还靠MySQL完成。

```
package com.viewscenes.netsupervisor.spi  
  
public class MyDriver extends NonRegisteringDriver implements Driver{  
    static {  
        try {  
            java.sql.DriverManager.registerDriver(new MyDriver());  
        }  
    }  
}
```

```

        } catch (SQLException E) {
            throw new RuntimeException("Can't register driver!");
        }
    }

    public MyDriver() throws SQLException {}

    public Connection connect(String url, Properties info) throws SQLException {
        System.out.println("准备创建数据库连接.url:"+url);
        System.out.println("JDBC配置信息: "+info);
        info.setProperty("user", "root");
        Connection connection = super.connect(url, info);
        System.out.println("数据库连接创建完成!" + connection.toString());
        return connection;
    }
}

```

-----输出结果-----

准备创建数据库连接.url:jdbc:mysql:///consult?serverTimezone=UTC

JDBC配置信息: {user=root, password=root}

数据库连接创建完成!com.mysql.cj.jdbc.ConnectionImpl@7cf10a6f